Structural and Narrative Dependencies

Structural dependencies encode the machine-readable edges that determine how one node in the outline relies on another. In the canonical work object, these edges are explicit data rather than implied reading order: a section can declare that it *builds on* a prior chapter, that it *depends on* a specific subsection, or that it is otherwise constrained by another part of the tree. This makes the outline actionable for validation, scheduling, and generation, because the system can inspect the dependency graph before it assigns work or compiles output.

The practical value of structural dependencies is that they separate *where content lives* from *what must already exist*. A node may be nested under a chapter for organizational purposes while still depending on material introduced elsewhere. That distinction allows the outline to remain recursive and uniform without forcing the author to encode every relationship as a parent–child relationship. In other words, the tree gives the book its shape, while dependency edges give the system its execution order.

A simple example makes the distinction concrete. A subsection may appear inside Chapter 2 because it belongs in that part of the book, yet its structural dependency may point back to Chapter 1 if it relies on terminology or a conceptual frame established there. The nesting tells the reader where the subsection belongs in the manuscript; the dependency edge tells the machine when it is safe to generate, validate, or revise that subsection.

Narrative dependencies serve a different purpose. They are human-readable notes that explain why a section comes next, what conceptual bridge it provides, or what assumptions the reader is expected to carry forward. Where structural dependencies are optimized for machines, narrative dependencies are optimized for authors and reviewers. They help answer questions such as: What prior idea does this subsection rely on? What terminology has already been established? What transition does this section provide between two larger topics?

Used together, the two forms of dependency keep the outline both precise and legible. Structural edges support automated checks, task planning, and safe generation order. Narrative notes preserve authorial intent and make the outline easier to read as a design document. The result is a dependency model that is simultaneously operational and explanatory: one layer tells the system what must happen, and the other tells the reader why the sequence makes sense.